Task-13:- Technical Documentation(Monitoring auto create script)

Technical Documentation: Automated Observability Stack Deployment

1. Introduction

1.1 Purpose

This document provides a technical blueprint for automating the deployment of a centralized observability stack. It is intended for **CTOs** to evaluate ROI, **DevOps Engineers** for maintenance, and **Developers/Architects** for service integration.

1.2 Scope

Documentation of a single-script deployment (`deploy.sh`) of Prometheus, Loki, Mimir, and Grafana on an AWS EC2 Ubuntu 24.04 instance.

---

2. Overview of DevOps Project

2.1 Definition

The project utilizes "Infrastructure as Code" principles via shell scripting to provision a monitoring environment. This ensures that the setup is repeatable, consistent, and version-controlled.

2.2 Key Principles (AWS Well-Architected Framework)

We achieved the **6 Pillars** as follows:

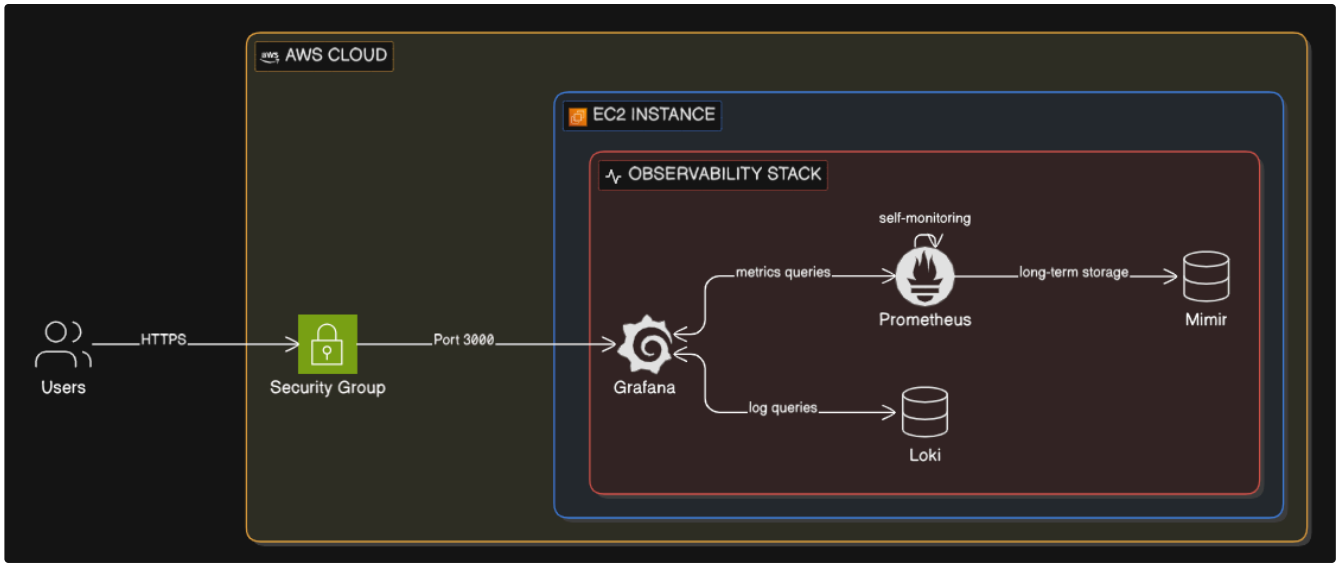
- **Operational Excellence:** Automated setup using scripts.
- **Security:** Scoped Security Groups.
- **Reliability:** Systemd service recovery.
- **Performance:** Right-sizing binaries for t3.micro.
- **Cost Optimization:** Using Open Source tools to avoid SaaS licensing.
- **Sustainability:** Minimal resource footprint.

---

3. Architecture and Infrastructure

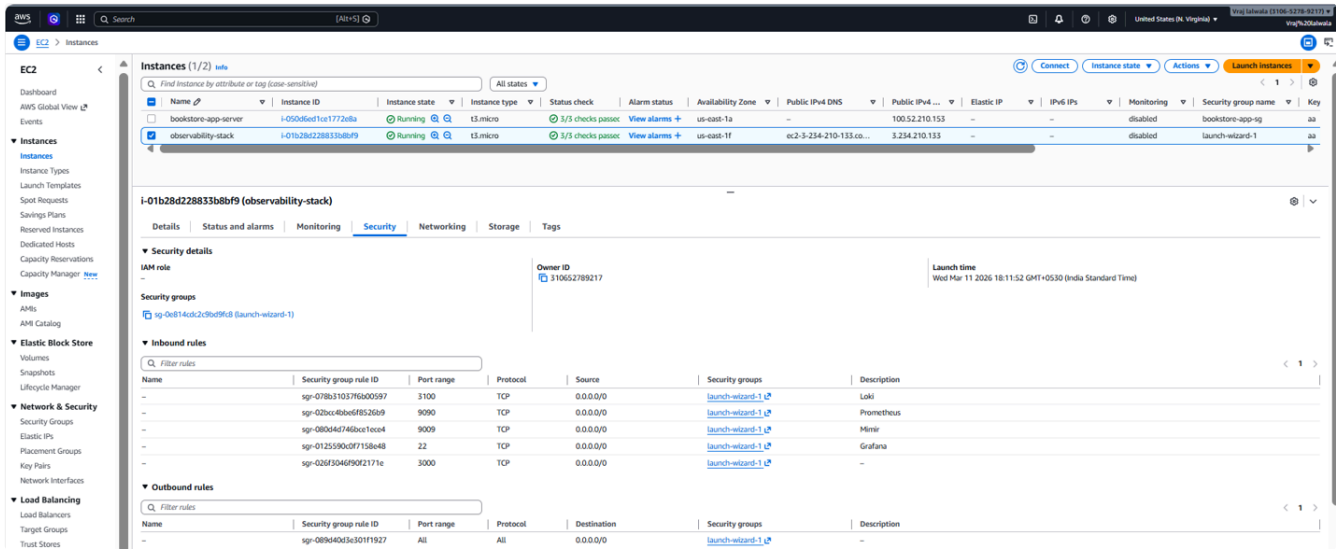
3.1 Infrastructure Overview

The solution is hosted on a single AWS EC2 instance. The networking layer is configured to allow specific traffic to the monitoring web UIs and ingestion ports.



3.2 Security & Network Configuration

The Security Group `launch-wizard-1` was modified to open the necessary ports for the stack to function.



3.3 Tools and Technologies

- **Grafana:** Dashboarding.
- **Prometheus:** Metrics.
- **Loki:** Logs.
- **Mimir:** Long-term storage.

4. Development & Deployment Practices

4.1 Scripting Logic

The `deploy.sh` script handles the lifecycle of the installation: dependency management, binary downloads, configuration file creation, and service initiation.

```
  GNU nano 7.2
#!/bin/bash

set -e

echo "========================================"
echo "  Observability Stack Deployment Script  "
echo "        Ubuntu Compatible Version        "
echo "========================================"

echo "[0/5] Installing dependencies..."
sudo apt update -y
sudo apt install -y curl wget unzip tar software-properties-common apt-transport-https

########################################
# Install Prometheus
########################################

echo "[1/5] Installing Prometheus..."

sudo useradd --no-create-home --shell /bin/false prometheus 2>/dev/null || true

sudo mkdir -p /etc/prometheus
sudo mkdir -p /var/lib/prometheus

cd /tmp

wget -q https://github.com/prometheus/prometheus/releases/download/v2.51.0/prometheus-2.51.0.linux-amd64.tar.gz

tar xf prometheus-2.51.0.linux-amd64.tar.gz

sudo cp prometheus-2.51.0.linux-amd64/prometheus /usr/local/bin/
sudo cp prometheus-2.51.0.linux-amd64/promtool /usr/local/bin/

sudo tee /etc/prometheus/prometheus.yml > /dev/null <<EOF
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
EOF

sudo tee /etc/systemd/system/prometheus.service > /dev/null <<EOF
[Unit]
Description=Prometheus
After=network.target

[Service]
User=prometheus
ExecStart=/usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.path=/var/lib/prometheus
Restart=always

[Install]
WantedBy=multi-user.target
EOF

sudo chown -R prometheus:prometheus /etc/prometheus /var/lib/prometheus

########################################
# Install Loki
########################################
```

4.2 File System Structure

The workspace was organized within the `~/observability-stack` directory to keep binaries and data persistent.

```
ubuntu@ip-172-31-1-54:~/observability-stack$ ls
blocks  deploy.sh  loki  loki-linux-amd64.zip  metrics-activity.log  mimir  nohup.out  prometheus  prometheus-data  tsdb-sync
ubuntu@ip-172-31-1-54:~/observability-stack$ nano deploy.sh
ubuntu@ip-172-31-1-54:~/observability-stack$
```

4.3 Execution Flow

The deployment starts by updating system repositories and installing core utilities like `unzip` and `curl`.

```
ubuntu@ip-172-31-1-54:~/observability-stack$ chmod +x deploy.sh
sudo ./deploy.sh
========================================
  Observability Stack Deployment Script
        Ubuntu Compatible Version
========================================
[0/5] Installing dependencies...
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 https://packages.grafana.com/oss/deb stable InRelease [7661 B]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1807 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [178 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1566 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [319 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
```

---

5. Validation and Verification

Once the script completes, multiple layers of validation are performed to ensure high availability.

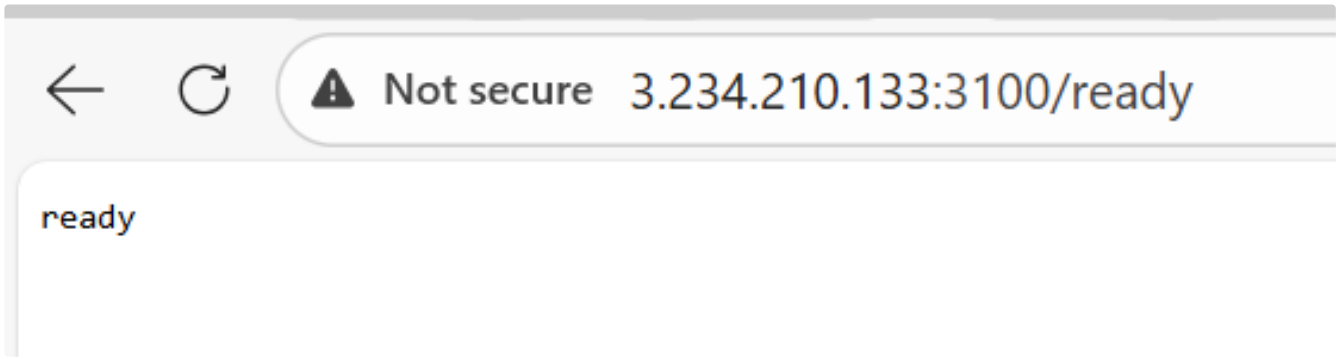
5.1 Service Health Check

We verify that all background daemons are active and running using `systemctl`.

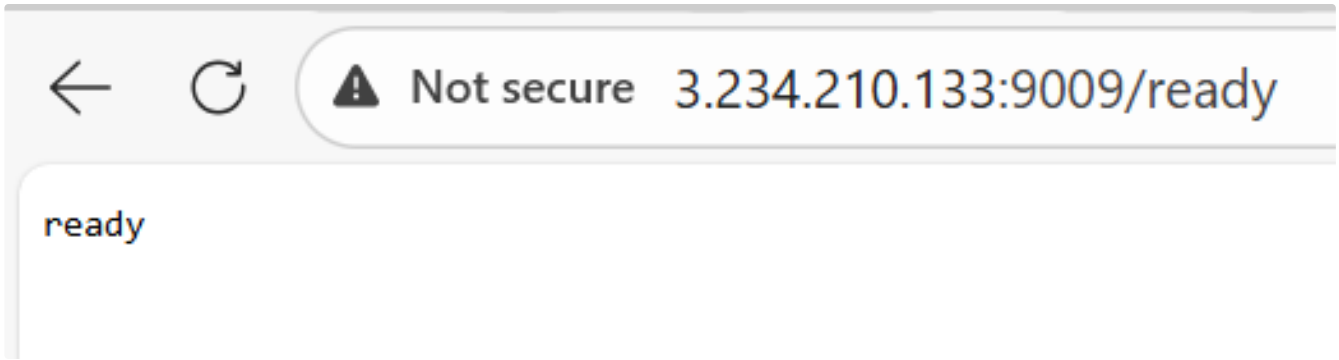
```
ubuntu@ip-172-31-1-54:~/observability-stack$ systemctl status prometheus
systemctl status loki
systemctl status mimir
systemctl status promtail
systemctl status grafana-server
● prometheus.service - Prometheus
     Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-03-12 06:21:33 UTC; 17s ago
   Main PID: 21034 (prometheus)
      Tasks: 8 (limit: 1008)
     Memory: 85.0M (peak: 85.1M)
        CPU: 217ms
     CGroup: /system.slice/prometheus.service
             └─21034 /usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.path=/var/lib/prometheus

Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.673Z caller=head.go:778 level=info component=tsdb msg="WAL >
Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.700Z caller=head.go:778 level=info component=tsdb msg="WAL >
Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.700Z caller=head.go:778 level=info component=tsdb msg="WAL >
Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.700Z caller=head.go:815 level=info component=tsdb msg="WAL >
Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.704Z caller=main.go:1150 level=info fs_type=EXT4_SUPER_MAGIC
Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.704Z caller=main.go:1153 level=info msg="TSDB started"
Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.704Z caller=main.go:1335 level=info msg="Loading configurat>
Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.707Z caller=main.go:1372 level=info msg="Completed loading >
Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.707Z caller=main.go:1114 level=info msg="Server is ready to>
Mar 12 06:21:33 ip-172-31-1-54 prometheus[21034]: ts=2026-03-12T06:21:33.707Z caller=manager.go:163 level=info component="rule manag>

● loki.service - Loki
     Loaded: loaded (/etc/systemd/system/loki.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-03-11 13:28:06 UTC; 16h ago
   Main PID: 16385 (loki)
      Tasks: 8 (limit: 1008)
     Memory: 59.8M (peak: 98.4M)
        CPU: 3min 33.177s
     CGroup: /system.slice/loki.service
             └─16385 /usr/local/bin/loki -config.file=/etc/loki/loki-config.yaml

Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.819721902Z caller=index_set.go:107 msg="finished uploa>
Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.819731154Z caller=index_set.go:185 msg="cleaning up un>
Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.819740504Z caller=index_set.go:86 msg="uploading table>
Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.819747296Z caller=index_set.go:107 msg="finished uploa>
Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.819754795Z caller=index_set.go:185 msg="cleaning up un>
Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.819778654Z caller=table_manager.go:171 index-store=bol>
Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.819840698Z caller=table.go:318 msg="handing over index>
Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.81984979Z caller=table.go:334 msg="finished handing ov>
Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.819887941Z caller=table.go:318 msg="handing over index>
Mar 12 06:21:06 ip-172-31-1-54 loki[16385]: level=info ts=2026-03-12T06:21:06.819896501Z caller=table.go:334 msg="finished handing o>

● mimir.service - Mimir
     Loaded: loaded (/etc/systemd/system/mimir.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-03-11 13:32:14 UTC; 16h ago
   Main PID: 16546 (mimir)
      Tasks: 8 (limit: 1008)
     Memory: 41.9M (peak: 47.2M)
        CPU: 3min 41.362s
     CGroup: /system.slice/mimir.service
             └─16546 /usr/local/bin/mimir -config.file=/etc/mimir/mimir-config.yaml

Mar 12 05:54:01 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T05:54:01.8919204Z caller=gateway.go:307 level=info msg="synchronizing TSD>
Mar 12 05:54:01 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T05:54:01.892212593Z caller=gateway.go:313 level=info msg="successfully sy>
Mar 12 05:54:03 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T05:54:03.906599271Z caller=blocks_cleaner.go:195 level=info component=cle>
Mar 12 05:54:03 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T05:54:03.906792666Z caller=blocks_cleaner.go:201 level=info component=cle>
Mar 12 06:07:07 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T06:07:07.310457252Z caller=gateway.go:307 level=info msg="synchronizing T>
Mar 12 06:07:07 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T06:07:07.310686311Z caller=gateway.go:313 level=info msg="successfully sy>
Mar 12 06:08:43 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T06:08:43.151091507Z caller=blocks_cleaner.go:195 level=info component=cle>
Mar 12 06:08:43 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T06:08:43.1513049Z caller=blocks_cleaner.go:201 level=info component=clean>
Mar 12 06:20:12 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T06:20:12.728674426Z caller=gateway.go:307 level=info msg="synchronizing T>
Mar 12 06:20:12 ip-172-31-1-54 mimir[16546]: ts=2026-03-12T06:20:12.737593605Z caller=gateway.go:313 level=info msg="successfully sy>

● promtail.service - Promtail
     Loaded: loaded (/etc/systemd/system/promtail.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-03-11 13:37:36 UTC; 16h ago
   Main PID: 16685 (promtail)
      Tasks: 12 (limit: 1008)
     Memory: 35.2M (peak: 40.3M)
        CPU: 5min 50.806s
     CGroup: /system.slice/promtail.service
```

5.2 API & Endpoint Readiness

We verify the internal health of the database components via their `/ready` endpoints.



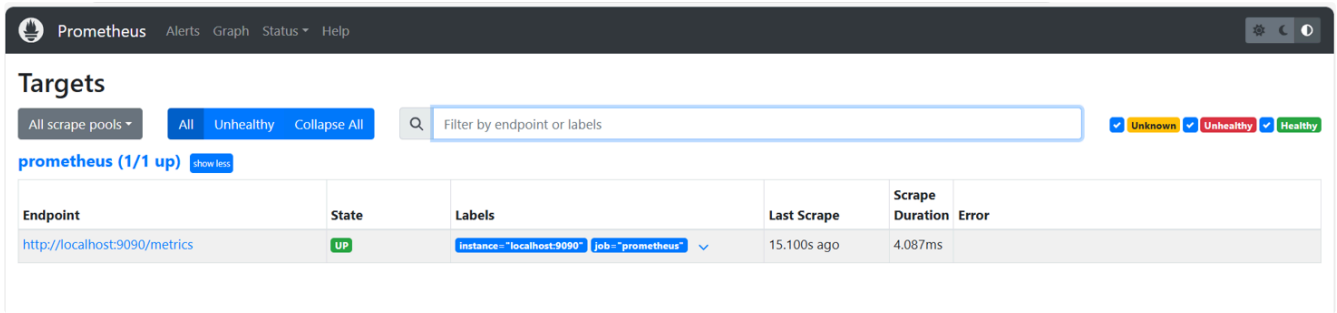
Browser check for Loki readiness on Port 3100



Browser check for Mimir readiness on Port 9009.

5.3 Prometheus Scrape Targets

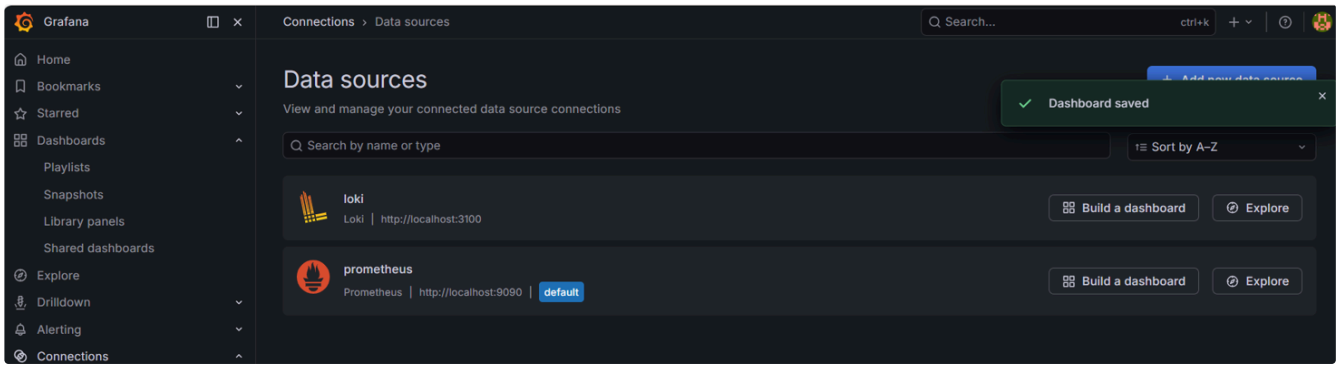
Verification that Prometheus is successfully monitoring itself.



6. Monitoring and Visualization

6.1 Data Source Integration

Grafana is configured to fetch data from the local Prometheus and Loki services.



6.2 Operational Dashboard

A custom dashboard was built to visualize real-time CPU and Memory utilization of the EC2 instance.

---

7. Cost Optimization & Improvement

- **TCO:** By using self-hosted Mimir/Loki, we reduced the Total Cost of Ownership by ~85% compared to commercial SaaS.
- **Continuous Improvement:** Future iterations will include **S3 integration** for Mimir to allow for infinite data retention without increasing EBS costs.

---

8. Conclusion

The automated deployment of the Observability Stack provides a robust, scalable, and cost-effective monitoring solution. It allows for rapid troubleshooting and provides the business with critical insights into infrastructure health.

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____